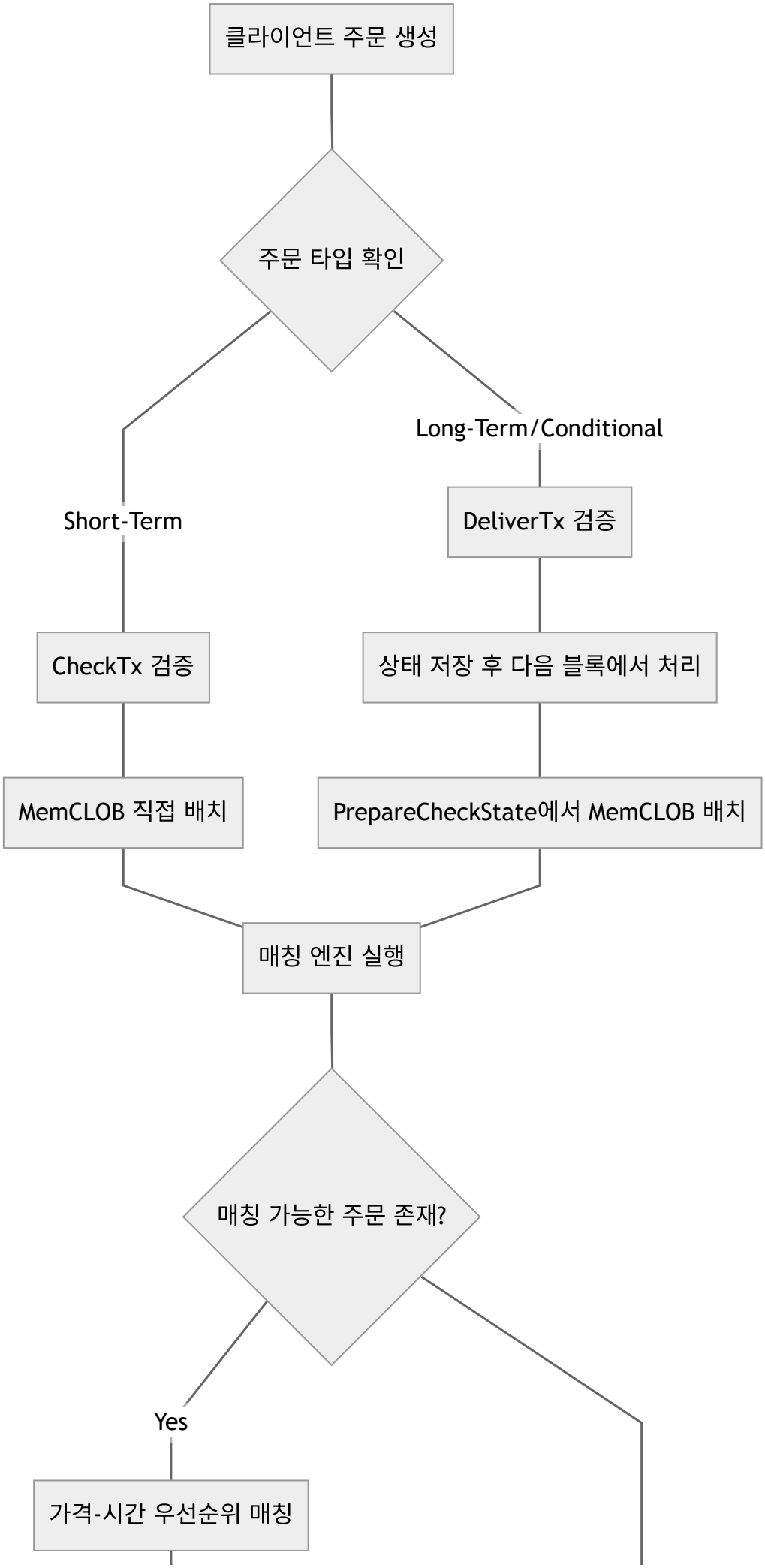
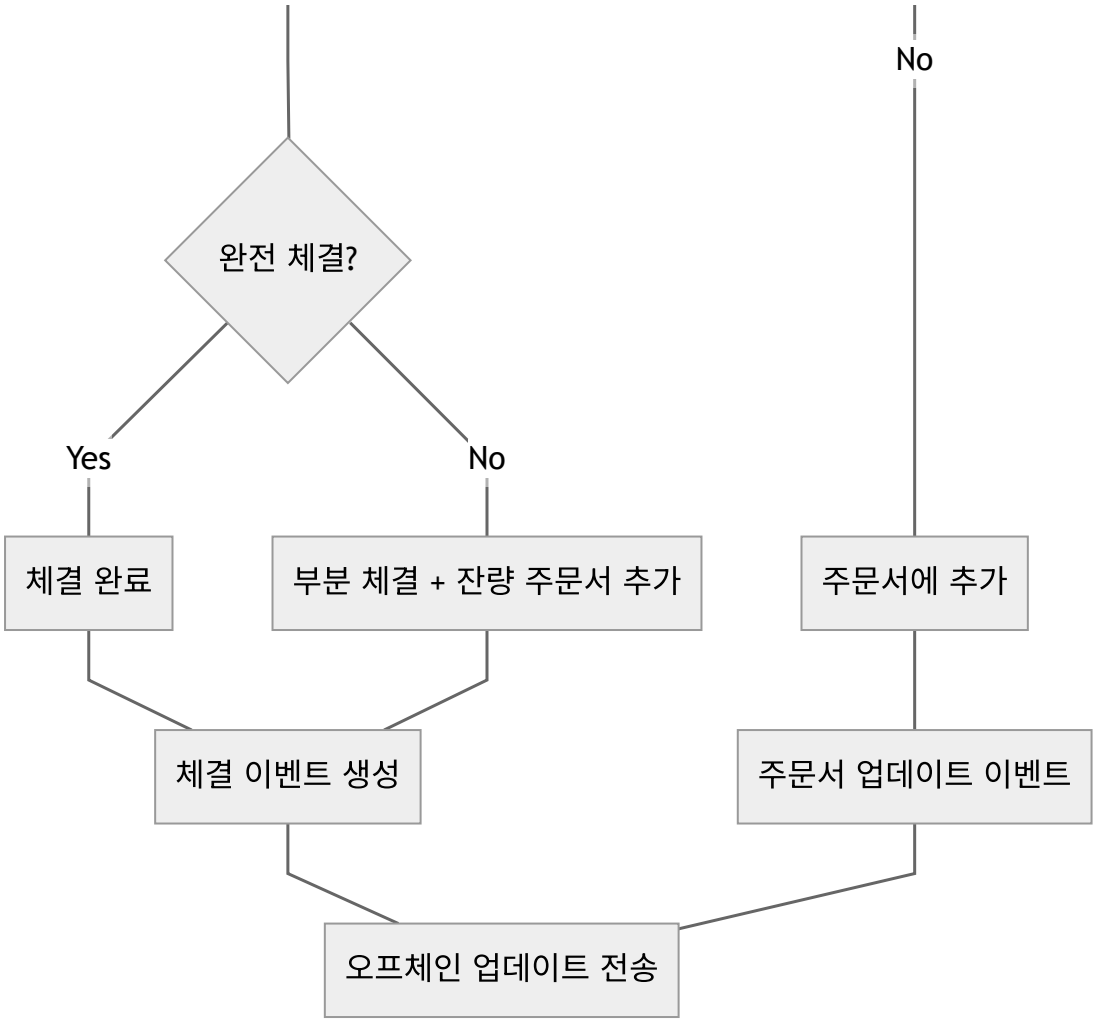
dYdX v4 거래 처리 플로우 - 주문 생성부터 매칭까지의 라이프사이클 분석

개요

이 문서는 dYdX v4에서 주문이 생성되어 매칭되기까지의 전체 라이프사이클을 상세히 분석합니다. 주문 검증 및 상태 관리 프로세스, 매칭 엔진의 가격-시간 우선순위 알고리즘 구현, 그리고 부분 체결 및 완전 체결 처리 로직을 포함합니다.

주문 라이프사이클 개요


클라이언트 주문 생성
주문 타입 확인
Short-Term
Long-Term/Conditional
CheckTx 검증
DeliverTx 검증
상태 저장 후 다음 블록에서 처리
MemCLOB 직접 배치
PrepareCheckState에서 MemCLOB 배치
매칭 엔진 실행
매칭 가능한 주문 존재?
Yes
가격-시간 우선순위 매칭



주문 검증 및 상태 관리 프로세스

1. 주문 타입별 검증 프로세스

Short-Term Orders (단기 주문)

특징:

- 메모리에만 저장되며 블록체인 상태에 기록되지 않음
- 블록 번호 기반 만료 (Good-Till-Block)
- CheckTx에서 즉시 처리

검증 과정:

```
// CheckTx 단계에서의 Short-Term 주문 처리
func (k Keeper) PlaceShortTermOrder(ctx  sdk.Context, msg *types.MsgPlaceOrder)
```

```
order := msg.GetOrder()

// 1. 기본 주문 검증

if err := k.validateOrder(ctx, order); err != nil {

return err

}

// 2. 속도 제한 검사

if err := k.RateLimitPlaceOrder(ctx, msg); err != nil {

return err

}

// 3. 서브계정 검증 (잔고, 마진 등)

if err := k.validateSubaccount(ctx, order); err != nil {

return err

}

// 4. MemCLOB에 직접 배치 및 매칭 시도

_, orderStatus, offchainUpdates, err := k.MemClob.PlaceOrder(ctx, order)

// 5. 오프체인 업데이트 전송

k.SendOffchainMessages(offchainUpdates, nil, metrics.SendPlaceOrderOffchainUpdat

return err

}
```

Long-Term Orders (장기 주문)

특징:

- 블록체인 상태에 영구 저장

- 시간 기반 만료 (Good-Till-Time)
- DeliverTx에서 상태 저장 후 다음 블록에서 처리

처리 과정:

```
// DeliverTx 단계에서의 Long-Term 주문 처리
func (k Keeper) HandleMsgPlaceOrder(ctx  sdk.Context, msg *types.MsgPlaceOrder)
order := msg.GetOrder()
// 1. Stateful 주문 검증
if  err := k.validateStatefulOrder(ctx, order); err != nil {
return  err
}
// 2. 상태에 주문 배치 정보 저장
k.SetLongTermOrderPlacement(ctx, order, ctx.BlockHeight())
// 3. ProcessProposerMatchesEvents 업데이트
k.MustUpdateProcessProposerMatchesEvents(ctx, order)
// 4. 온체인 인덱서 이벤트 생성
k.GetIndexerEventManager().AddTxnEvent(
ctx,
indexerevents.SubtypeStatefulOrder,
indexerevents.StatefulOrderEventVersion,
indexer_manager.GetBytes(
indexerevents.NewStatefulOrderPlacementEvent(order),
),
```

```
)
return nil
}
```

Conditional Orders (조건부 주문)

특징:

- 특정 조건 만족 시에만 활성화
- 트리거 가격 기반 활성화
- 복잡한 거래 전략 지원

트리거 메커니즘:

```
// EndBlocker에서 조건부 주문 트리거 확인
func (k Keeper) MaybeTriggerConditionalOrders(ctx sdk.Context) []types.OrderId
var triggeredOrderIds []types.OrderId
// 모든 CLOB 쌍에 대해 조건부 주문 확인
for clobPairId, untriggeredOrders := range k.UntriggeredConditionalOrders {
// 현재 시장 가격 가져오기
marketPrice, err := k.pricesKeeper.GetMarketPrice(ctx, clobPairId.ToUint32())
if err != nil {
continue
}
// 트리거 조건 확인
for orderId, conditionalOrder := range untriggeredOrders.Orders {
if k.shouldTriggerConditionalOrder(conditionalOrder, marketPrice) {
```

```
// 주문을 Long-Term 주문으로 변환하여 상태에 저장
k.SetLongTermOrderPlacement(ctx, conditionalOrder.Order, ctx.BlockHeight())
triggeredOrderIds = append(triggeredOrderIds, orderId)
// 트리거되지 않은 주문 목록에서 제거
delete(untriggeredOrders.Orders, orderId)
}
}
}
return triggeredOrderIds
}
```

2. 주문 상태 관리

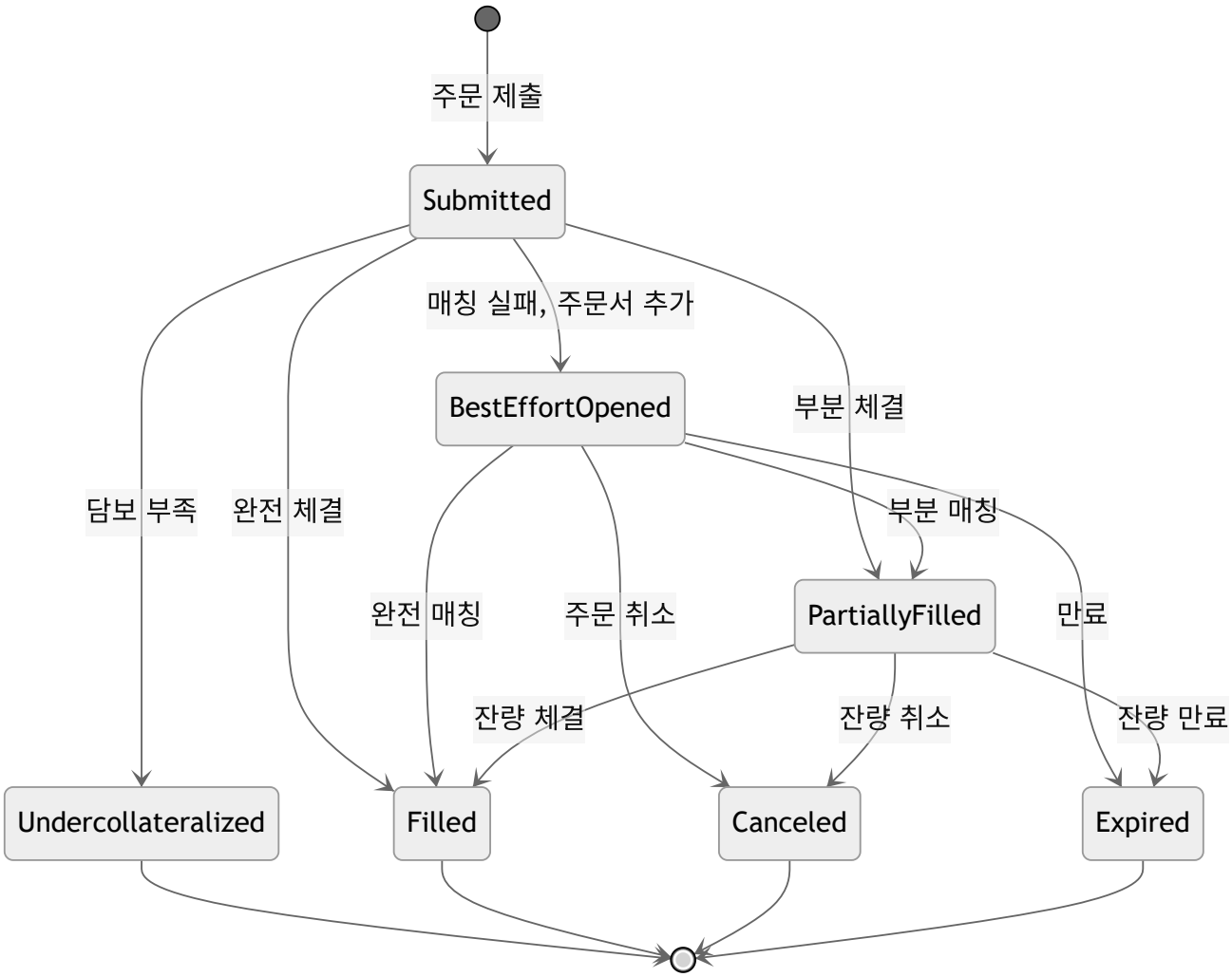
주문 상태 종류

```
type OrderStatus uint32

const (
// 성공적으로 배치됨
Success OrderStatus = 0
// 담보 부족으로 거부됨
Undercollateralized OrderStatus = 1
// 주문서에 추가됨 (매칭되지 않음)
BestEffortOpened OrderStatus = 2
```

```
// 부분적으로 체결됨
PartiallyFilled  OrderStatus = 3
// 완전히 체결됨
Filled  OrderStatus = 4
// 취소됨
Canceled  OrderStatus = 5
)
```

상태 전이 다이어그램



매칭 엔진의 가격-시간 우선순위 알고리즘 구현 분석

1. 매칭 우선순위 규칙

가격 우선순위 (Price Priority)

```
// 매수 주문: 높은 가격이 우선
func (o *OrderbookSide) getBestBidPrice() types.Subticks {
if len(o.LevelKeysDesc) == 0 {
return 0
}
return o.LevelKeysDesc[0] // 내림차순 정렬된 첫 번째 가격
}

// 매도 주문: 낮은 가격이 우선
func (o *OrderbookSide) getBestAskPrice() types.Subticks {
if len(o.LevelKeysAsc) == 0 {
return math.MaxUint64
}
return o.LevelKeysAsc[0] // 오름차순 정렬된 첫 번째 가격
}
```

시간 우선순위 (Time Priority)

```
// 같은 가격 레벨 내에서 시간 순서로 매칭
```

```
type Level struct {
// 시간 순서로 정렬된 주문 리스트
Orders *list.List
// 해당 가격 레벨의 총 수량
TotalQuantums satypes.BaseQuantums
}

// 주문을 시간 순서대로 레벨에 추가
func (l *Level) addOrder(order types.Order) *list.Element {
return l.Orders.PushBack(order) // 리스트 끝에 추가 (FIFO)
}
```

2. 매칭 알고리즘 구현

핵심 매칭 로직

```
func (m *MemClobPriceTimePriority) PlaceOrder(
ctx sdk.Context,
order types.Order,
) (
orderSizeOptimisticallyFilledFromMatchingQuantums satypes.BaseQuantums,
orderStatus types.OrderStatus,
offchainUpdates *types.OffchainUpdates,
err error,
```

```
) {
// 1. 주문 검증
if err := m.validatePlaceOrder(ctx, order); err != nil {
return 0, types.Undercollateralized, nil, err
}
// 2. 매칭 시도
matchedQuantums, remainingOrder, matches, err := m.attemptMatching(ctx, order)
if err != nil {
return 0, types.Undercollateralized, nil, err
}
// 3. 매칭 결과 처리
for _, match := range matches {
m.processMatch(ctx, match)
}
// 4. 남은 주문을 주문서에 추가
if remainingOrder != nil {
m.addToOrderbook(ctx, remainingOrder)
orderStatus = types.BestEffortOpened
} else {
orderStatus = types.Filled
}
// 5. 오프체인 업데이트 생성
offchainUpdates = m.generateOffchainUpdates(ctx, order, matches)
```

```
return matchedQuantums, orderStatus, offchainUpdates, nil
}
```

매칭 시도 알고리즘

```
func (m *MemClobPriceTimePriority) attemptMatching(
ctx sdk.Context,
order types.Order,
) (
matchedQuantums satypes.BaseQuantums,
remainingOrder *types.Order,
matches []types.MatchedOrder,
err error,
) {
orderbook := m.openOrders.orderbooksMap[order.GetClobPairId()]
oppositeSide := orderbook.getOppositeSide(order.GetOrderSide())
remainingQuantums := order.GetQuantums()
orderPrice := order.GetSubticks()
// 반대편 주문서에서 매칭 가능한 주문들을 가격 순서대로 처리
for _, priceLevel := range oppositeSide.getSortedPriceLevels() {
// 가격 조건 확인
if !m.canMatch(order.GetOrderSide(), orderPrice, priceLevel) {
break // 더 이상 매칭 불가능
```

```
}

// 해당 가격 레벨의 주문들을 시간 순서대로 매칭

level := oppositeSide.Levels[priceLevel]

for  element := level.Orders.Front(); element != nil; {

makerOrder := element.Value.(types.Order)

next := element.Next()

// 매칭 수량 계산

matchQuantums := min(remainingQuantums, makerOrder.GetQuantums())

// 매칭 실행

match := types.MatchedOrder{

MakerOrder: makerOrder,

TakerOrder: order,

MatchQuantums: matchQuantums,

MatchPrice: priceLevel,

}

matches = append(matches, match)

// 수량 업데이트

remainingQuantums -= matchQuantums

matchedQuantums += matchQuantums

// Maker 주문 처리

if  makerOrder.GetQuantums() == matchQuantums {

// 완전 체결 - 주문서에서 제거

m.removeOrderFromLevel(level, element)
```

```
} else {
// 부분 체결 - 잔량 업데이트
m.updateOrderQuantums(element, makerOrder.GetQuantums()-matchQuantums)
}
// Taker 주문이 완전 체결되면 매칭 종료
if  remainingQuantums == 0 {
return  matchedQuantums, nil, matches, nil
}
element = next
}
}
// 잔량이 있으면 남은 주문 생성
if  remainingQuantums > 0 {
remainingOrder = &types.Order{
OrderId: order.GetOrderId(),
Side: order.GetOrderSide(),
Quantums: remainingQuantums,
Subticks: order.GetSubticks(),
// ... 기타 필드 복사
}
}
return  matchedQuantums, remainingOrder, matches, nil
```

```
}
```

3. 가격 매칭 조건

```
// 매칭 가능 여부 확인
func (m *MemClobPriceTimePriority) canMatch(
takerSide  types.Order_Side,
takerPrice  types.Subticks,
makerPrice  types.Subticks,
) bool {
if  takerSide == types.Order_SIDE_BUY {
// 매수 주문: taker 가격이 maker 가격보다 높거나 같아야 함
return  takerPrice >= makerPrice
} else {
// 매도 주문: taker 가격이 maker 가격보다 낮거나 같아야 함
return  takerPrice <= makerPrice
}
}
```

부분 체결 및 완전 체결 처리 로직

1. 체결 결과 처리

체결 정보 생성

```
type Fill struct {
// 체결 수량
FillAmount satypes.BaseQuantums
// Maker/Taker 주문 ID
MakerOrderId types.OrderId
TakerOrderId types.OrderId
// 수수료
MakerFee int64
TakerFee int64
// 체결 가격
Price types.Subticks
// 체결 시간
BlockHeight uint32
}
```

체결 처리 과정

```
func (m *MemClobPriceTimePriority) processMatch(
ctx sdk.Context,
match types.MatchedOrder,
) error {
// 1. 체결 정보 생성
fill := types.Fill{
```

```
FillAmount: match.MatchQuantums,
MakerOrderId: match.MakerOrder.GetOrderId(),
TakerOrderId: match.TakerOrder.GetOrderId(),
Price: match.MatchPrice,
BlockHeight: lib.MustConvertIntegerToUint32(ctx.BlockHeight()),
}
// 2. 수수료 계산
makerFee, takerFee := m.calculateFees(ctx, match)
fill.MakerFee = makerFee
fill.TakerFee = takerFee
// 3. 서브계정 업데이트
if err := m.updateSubaccountsForFill(ctx, fill); err != nil {
return err
}
// 4. 체결 이벤트 생성
m.generateFillEvent(ctx, fill)
return nil
}
```

2. 부분 체결 처리

주문 분할 로직

```
func (m *MemClobPriceTimePriority) handlePartialFill(
```

```
ctx  sdk.Context,
originalOrder  types.Order,
filledQuantums  satypes.BaseQuantums,
) (*types.Order, error) {
remainingQuantums := originalOrder.GetQuantums() - filledQuantums
if  remainingQuantums == 0 {
// 완전 체결 - 남은 주문 없음
return  nil, nil
}
// 부분 체결 - 잔량으로 새 주문 생성
remainingOrder := types.Order{
OrderId: originalOrder.GetOrderId(),
Side: originalOrder.GetOrderSide(),
Quantums: remainingQuantums,
Subticks: originalOrder.GetSubticks(),
TimeInForce: originalOrder.GetTimeInForce(),
ReduceOnly: originalOrder.GetReduceOnly(),
ClientId: originalOrder.GetClientId(),
}
return &remainingOrder, nil
}
```

잔량 주문서 추가

```go
func (m *MemClobPriceTimePriority) addRemainingOrderToBook(
ctx  sdk.Context,
remainingOrder *types.Order,
) error {
if  remainingOrder == nil {
return  nil
}
// 1. 주문서에서 적절한 가격 레벨 찾기 또는 생성
orderbook := m.openOrders.orderbooksMap[remainingOrder.GetClobPairId()]
side := orderbook.getSide(remainingOrder.GetOrderSide())
level := side.getOrCreateLevel(remainingOrder.GetSubticks())
// 2. 주문을 레벨에 추가 (시간 순서 유지)
element := level.addOrder(*remainingOrder)
// 3. 빠른 검색을 위한 맵 업데이트
m.openOrders.orderIdToLevelOrder[remainingOrder.GetOrderId()] = element
// 4. 서브계정 주문 추적 업데이트
m.updateSubaccountOrders(*remainingOrder, true)
// 5. 주문서 업데이트 이벤트 생성
m.generateOrderbookUpdateEvent(ctx, *remainingOrder)
return  nil
}
```

3. 완전 체결 처리

주문 완료 처리

```
func (m *MemClobPriceTimePriority) handleCompleteFill(
ctx  sdk.Context,
order  types.Order,
totalFilledQuantums  satypes.BaseQuantums,
) error {
// 1. 주문이 주문서에 있었다면 제거
if  element, exists := m.openOrders.orderIdToLevelOrder[order.GetOrderId()]; exi
m.removeOrderFromOrderbook(ctx, element)
}
// 2. 완전 체결 이벤트 생성
m.generateOrderCompleteEvent(ctx, order, totalFilledQuantums)
// 3. 서브계정 주문 추적에서 제거
m.updateSubaccountOrders(order, false)
return  nil
}
```

주문 검증 세부 사항

1. 기본 주문 검증

```
func (k Keeper) validateOrder(ctx  sdk.Context, order  types.Order) error {
```

```
// 1. CLOB 쌍 유효성 확인
clobPair, exists := k.GetClobPair(ctx, order.GetClobPairId())
if !exists {
return  types.ErrInvalidClob
}
// 2. 주문 크기 검증
if  order.GetQuantums() == 0 {
return  types.ErrInvalidOrderSize
}
// 3. 가격 검증
if  order.GetSubticks() == 0 {
return  types.ErrInvalidOrderPrice
}
// 4. 만료 시간 검증
if  order.IsExpired(ctx.BlockHeight(), ctx.BlockTime()) {
return  types.ErrOrderExpired
}
// 5. 주문 ID 중복 확인
if  k.orderExists(ctx, order.GetOrderId()) {
return  types.ErrOrderAlreadyExists
}
return  nil
```

```
}
```

2. 서브계정 검증

```
func (k Keeper) validateSubaccount(ctx  sdk.Context, order  types.Order) error {
subaccountId := order.GetOrderId().GetSubaccountId()
// 1. 서브계정 존재 확인
subaccount := k.subaccountsKeeper.GetSubaccount(ctx, subaccountId)
// 2. 잔고 확인
if !k.hassufficientBalance(ctx, subaccount, order) {
return  types.ErrInsufficientBalance
}
// 3. 마진 요구사항 확인
if !k.meetsMargingRequirements(ctx, subaccount, order) {
return  types.ErrInsufficientMargin
}
// 4. 포지션 한도 확인
if !k.withinPositionLimits(ctx, subaccount, order) {
return  types.ErrPositionLimitExceeded
}
return  nil
}
```

성능 최적화 기법

1. 메모리 최적화

- **객체 풀링**: 자주 생성/삭제되는 주문 객체 재사용
- **지연 삭제**: 즉시 삭제 대신 마킹 후 배치 삭제
- **압축된 데이터 구조**: 메모리 사용량 최소화

2. 알고리즘 최적화

- **이진 검색**: 가격 레벨 검색 최적화
- **배치 처리**: 여러 매칭을 한 번에 처리
- **지연 평가**: 필요할 때만 계산 수행

3. 동시성 최적화

- **읽기 최적화**: 주문서 조회의 병렬 처리
- **락 최소화**: 세밀한 락 전략으로 경합 감소
- **원자적 연산**: 경쟁 조건 방지

결론

dYdX v4의 주문 라이프사이클은 다양한 주문 타입을 지원하면서도 고성능을 유지하도록 설계되었습니다. 가격-시간 우선순위 기반의 매칭 엔진과 효율적인 부분 체결 처리 로직을 통해 기존 중앙화 거래소에 필적하는 성능과 기능을 제공합니다. 특히 MemCLOB의 인메모리 처리와 체계적인 주문 상태 관리는 높은 처리량과 낮은 지연시간을 달성하는 핵심 요소입니다.